

# ЗВІТ ДО ЛАБОРАТОРНОЇ РОБОТИ 1

Вибір та реалізація базових фреймворків та бібліотек

Індивідуальний варіант А

Backend-сервіс

Мова програмування Python

Виконала: Гончарова Наталя

Група: ФБ-51мн

## 1 Мета роботи

Дослідити бібліотеки реалізації криптографічних механізмів, визначити критерії їх вибору для різних програмно-апаратних платформ і набути практичних навичок коректного використання криптографічних API. Порівняти cryptography та PyCryptodome для Python, виконати автентифіковане шифрування й цифровий підпис, обґрунтувати рішення для backend-сервісу .

## 2 Принципи реалізації криптографічних механізмів

### 2.1 Використання готових реалізацій

Самостійна реалізація стандартного алгоритму може давати правильний результат на прикладах, але містити витoki через час виконання, помилки перевірки вхідних даних, роботи з пам'яттю або випадковими значеннями. Підтримувана бібліотека дає перевірені інтерфейси та механізм виправлення помилок. Водночас застосунок повинен правильно обирати режим, керувати ключами та перевіряти автентичність; сама наявність бібліотеки не гарантує безпеку .

### 2.2 Групи примітивів і допоміжних функцій

Типові групи: симетричне шифрування та AEAD; асиметричне шифрування й узгодження ключів; цифрові підписи; геш-функції та MAC; функції формування ключів KDF. Допоміжні засоби охоплюють генерацію

випадкових значень, імпорт та експорт ключів, серіалізацію, доповнення блоків і роботу із сертифікатами. Приклади: AES-GCM, RSA-OAEP, X25519, Ed25519, SHA-256, HMAC і HKDF .

### 2.3 Примітив і протокол

Криптографічний примітив виконує окрему базову функцію: наприклад, SHA-256 обчислює геш, AES шифрує блок, Ed25519 формує підпис. Протокол визначає послідовність дій сторін, формат повідомлень, правила перевірки та реакцію на помилки. Наприклад, TLS поєднує узгодження ключів, автентифікацію та захист трафіку. Коректні примітиви не виправляють помилки протоколу, зокрема відсутність перевірки особи співрозмовника .

### 2.4 Криптографічна випадковість

CSPRNG є генератором, для якого передбачення наступних значень без знання внутрішнього стану має бути обчислювально нездійсненним. Його початковий стан потребує достатньої ентропії. Генератор модуля `random` призначений для моделювання й не має необхідних гарантій для ключів. У Python застосовують `secrets` або `os.urandom`, що використовують системне джерело випадковості; час запуску чи фіксоване число не є безпечним `seed`.

### 2.5 Рівні криптографічного API

Високорівневий API приховує частину небезпечних налаштувань: прикладом є `Fernet` у `cryptography`. Низькорівневий API дозволяє окремо обирати примітиви, режими, `padding` і параметри, тому помилки можуть спричинити повтор `nonce`, використання ECB або прийняття неавтентифікованих даних. Клас `AESGCM` належить до шару `hazmat`, хоча об'єднує шифрування та перевірку `tag` в одному інтерфейсі .

### 2.6 Постійний час виконання

`Constant-time` означає, що гілки виконання та доступ до пам'яті не повинні залежати від секретних значень у спосіб, доступний спостерігачу. Це зменшує ризик часових і пов'язаних кеш-атак; абсолютної однаковості тривалості через планувальник ОС не вимагається. Для порівняння MAC

застосовують спеціалізовані функції, наприклад `secrets.compare_digest`. Такий підхід не є універсальним захистом від усіх фізичних побічних каналів.

## 2.7 Стандарт FIPS 140-3

FIPS 140-3 встановлює вимоги безпеки криптографічних модулів, зокрема до інтерфейсів, керування секретними параметрами, самотестування та фізичного захисту. Валідація може бути потрібна для державних закупівель або договірних вимог замовника. Вона стосується конкретного модуля, версії, середовища та режиму використання; підтримка AES чи підключення OpenSSL самі по собі не роблять Python-застосунок сертифікованим.

## 2.8 Арифметика багаторозрядних чисел

RSA оперує модулями й показниками довжиною в тисячі біт, які не вміщуються в звичайний машинний тип. Потрібні багаторозрядні множення, ділення, модульне піднесення до степеня та обернення. Крім швидкодії, важливі правильність на граничних значеннях, перевірка ключів, відсутність секретозалежних часових витоків, безпечна робота з пам'яттю та протидія збоям. Для приватних операцій застосовують, зокрема, засліплення; звичайний Python `int` не слід вважати constant-time реалізацією RSA.

## 3 Порівняння бібліотек для Python

Порівняння виконано за офіційною документацією, перевіреною 16.09.2026. Для дослідів використано `cryptography 50.0.1`. Останній завершений випуск, зазначений у переглянутому журналі `PyCryptodome`, – 3.23.0; розділ 3.24.0 позначено як розроблюваний. У таблиці наведено приклади можливостей, а не вичерпні переліки алгоритмів.

Таблиця 1 – Порівняння `cryptography` та `PyCryptodome`

| Критерій              | <code>cryptography</code>                             | <code>PyCryptodome</code>                         |
|-----------------------|-------------------------------------------------------|---------------------------------------------------|
| Мови та платформи     | Python 3.9+, PyPy; Linux, Windows 64-bit, macOS ARM64 | Python, PyPy; Linux, Windows, macOS; ядро на C    |
| Симетричне шифрування | AES, ChaCha20; вибір режиму через API                 | AES, ChaCha20, Salsa20 та сумісні старі алгоритми |

|                              |                                                                  |                                                                         |
|------------------------------|------------------------------------------------------------------|-------------------------------------------------------------------------|
| AEAD                         | AES-GCM, AES-CCM, ChaCha20-Poly1305                              | AES-GCM, CCM, EAX, SIV, OCB; ChaCha20-Poly1305                          |
| Асиметрична криптографія     | RSA-OAEP; ECDH, X25519 і X448                                    | RSA-OAEP; ECDH, X25519 і X448                                           |
| Цифровий підпис              | RSA-PSS, ECDSA, Ed25519, Ed448                                   | RSA-PSS, ECDSA, EdDSA                                                   |
| Геш-функції та KDF           | SHA-2, SHA-3; HKDF, PBKDF2, scrypt                               | SHA-2, SHA-3, BLAKE2; HKDF, PBKDF2, scrypt                              |
| Ініціалізація випадковістю   | Системна ентропія; ручний seed не потрібний                      | Випадковість ОС через os.urandom; власний seed не потрібний             |
| Отримання випадкових значень | Генератори ключів; os.urandom і secrets для байтів               | Crypto.Random.get_random_bytes(n)                                       |
| Високорівневий API           | Є recipes, зокрема Fernet; AESGCM розташований у hazmat          | API примітивів і допоміжних протоколів; параметри контролює розробник   |
| Апаратне прискорення         | Через криптографічний backend; залежить від CPU і збірки         | AES-NI на сумісних x86; оптимізовані модулі на C                        |
| FIPS                         | Лише з валідованим модулем і належною конфігурацією              | Валідацію пакета в документації не заявлено                             |
| Ліцензія                     | Apache License 2.0 або BSD, на вибір                             | BSD 2-Clause та public domain                                           |
| Вразливості та виправлення   | PKCS#7 oracle усунуто в 50.0.0; OpenSSL оновлено в 50.0.1        | CVE-2023-52323: побічний канал RSA-OAEP; виправлення в 3.19.1           |
| Організації та спільнота     | PyCA; публічний процес розробки й повідомлення про вразливості   | Команда PyCryptodome та спільнота; внески, публічні виправлення й тести |
| Документація та підтримка    | Версійні довідники; security-підтримка main і найновішого релізу | Довідник API й changelog; оновлення 3.23.0 та гілка наступного випуску  |

Для практичної частини обрано cryptography: API AESGCM перевіряє цілісність під час розшифрування, а Ed25519 забезпечує короткий шлях генерації ключів, підписання й перевірки. Бібліотека має документацію щодо помилок і підтримки. PyCryptodome також придатна, особливо коли потрібний ширший вибір режимів. Перевагу за швидкодією в цій роботі не встановлено, оскільки порівняльний бенчмарк не проводився.

Наведені вразливості стосуються конкретних реалізацій і версій; вони не доводять злам AES або Ed25519. Практична вимога – перевіряти залежності та своєчасно оновлювати пакети. Для wheels cryptography слід враховувати вбудовану версію OpenSSL: оновлення системної бібліотеки не обов’язково оновлює її копію в пакеті.

#### 4 Практичне використання cryptography

Дослід виконано у Python 3.12.14 із cryptography 50.0.1. Файл lab1.py містить обидва контрольні приклади та автоматичну перевірку очікуваних результатів. Для відтворення потрібно встановити залежність і запустити програму з каталогу, у якому розміщені файли:

```
python -m pip install -r requirements.txt  
python lab1.py --output-dir results
```

Програма зберігає текстові результати, JSON та HTML-сторінки. На рисунках 1 і 2 наведено скриншоти HTML-сторінок із фактичним виводом виконаного запуску. Секретні ключі не виводяться й не записуються у файли. Кожний запуск генерує нові ключі, тому значення шифротексту та підпису змінюються.

##### 4.1 Автентифіковане шифрування AES-GCM

Ключ AES-256 має 32 байти, nonce – 12 байтів, tag – 16 байтів. Ключ є секретом; nonce передається разом із шифротекстом і має бути унікальним для цього ключа. Tag перевіряє цілісність шифротексту й пов’язаних даних AAD. AAD не шифрується; у прикладі містить ідентифікатор запису та версію формату.

Основний фрагмент шифрування та перевірки пошкоджених даних:

```

from secrets import token_bytes
from cryptography.exceptions import InvalidTag
from cryptography.hazmat.primitives.ciphers.aead import AESGCM

key = AESGCM.generate_key(bit_length=256)
nonce = token_bytes(12)
aad = b"lab1|record=42|version=1"
message = b"Confidential user data: user_id=42"
aes = AESGCM(key)
encrypted = aes.encrypt(nonce, message, aad)
restored = aes.decrypt(nonce, encrypted, aad)
modified = bytearray(encrypted)
modified[0] ^= 1
try:
    aes.decrypt(nonce, bytes(modified), aad)
except InvalidTag:
    print("InvalidTag: modified ciphertext rejected")

```

```
(venv) ~/PycharmProjects/proj_1/1-2/Лабораторна_1_A_Комплект % python3 lab1.py --output-dir results
Python 3.12.0 | cryptography 50.0.1
Алгоритм: AES-256-GCM
Повідомлення: Confidential user data: user_id=42
Ключ: 256 біт (значення не виводиться)
Nonce (12 байт): eec7ea53a62b71932ef279de
AAD: lab1|record=42|version=1
Шифротекст (34 байт):
52f699f7f3806f05eef7dcb8334b0379d31a81eecbbbed939287fe9b0ba75fbfe39bd
Tag (16 байт): 11c99840dc536ede9b161b85a78098da
Відновлено: Confidential user data: user_id=42
Початкові дані: PASS
Зміна байта 0: 52 -> 53
Змінений шифротекст: InvalidTag — відхилено, PASS
```

## Автентифіковане шифрування

Python 3.12.0 | cryptography 50.0.1  
Алгоритм: AES-256-GCM  
Повідомлення: Confidential user data: user\_id=42  
Ключ: 256 біт (значення не виводиться)  
Nonce (12 байт): eec7ea53a62b71932ef279de  
AAD: lab1|record=42|version=1  
Шифротекст (34 байт):  
52f699f7f3806f05eef7dcb8334b0379d31a81eecbbbed939287fe9b0ba75fbfe39bd  
Tag (16 байт): 11c99840dc536ede9b161b85a78098da  
Відновлено: Confidential user data: user\_id=42  
Початкові дані: PASS  
Зміна байта 0: 52 -> 53  
Змінений шифротекст: InvalidTag — відхилено, PASS

### Рисунок 1 – Результат шифрування та перевірки AES-GCM

Повідомлення містить 34 байти. Метод `encrypt` повернув 50 байтів: 34 байти шифротексту та 16 байтів `tag`. Розшифрування повернуло початкові дані. Заміна першого байта з `ab` на `a7` залишила `tag` незмінним, тому повторне розшифрування завершилося `InvalidTag`. Це очікуване відхилення пошкодженого повідомлення; успішне виконання контролю не означає успішного розшифрування підміни.

Повтор пари ключ–nonce в GCM порушує конфіденційність і може дозволити підроблення `tag`. Випадковий nonce не дає математичної гарантії відсутності колізій. У цьому досліді під кожним новим ключем шифрується лише одне повідомлення; для багатопроцесного сервісу потрібна узгоджена

схема унікальності та обмеження використання ключа. Після InvalidTag дані не можна передавати подальшій бізнес-логіці .

## 4.2 Цифровий підпис Ed25519

Створено пару ключів Ed25519. Закритий ключ використано для підписання, відкритий – для перевірки; відкритий ключ у форматі Raw має 32 байти, підпис – 64 байти. Підпис підтверджує цілісність і походження повідомлення лише за умови, що відкритий ключ надійно пов’язаний із власником. Шифрування повідомлення цифровим підписом не виконується.

```
from cryptography.exceptions import InvalidSignature
from cryptography.hazmat.primitives.asymmetric import ed25519
```

```
message = b"Approve document 42"
private_key = ed25519.Ed25519PrivateKey.generate()
public_key = private_key.public_key()
signature = private_key.sign(message)
public_key.verify(signature, message)
modified = b"Approve document 43"
try:
    public_key.verify(signature, modified)
except InvalidSignature:
    print("InvalidSignature: modified message rejected")
```

```

Python 3.12.0 | cryptography 50.0.1
Алгоритм: Ed25519
Повідомлення: Approve document 42
Закритий ключ: створено, не експортується
Відкритий ключ (32 байт):
9b793e8533139a7ea88dd5f44b159e9a0e9898d9ea742e1da1d01773974777f4
Підпис (64 байт), hex у двох рядках:
e84bbf23b843c6b9adbbadb0110082cd6f9c4106aea7390cf6fd3714a69f56bb
fc6ab8c9025a5eefab28bacfebb744eb15399d0effdcfc9d4a36af658682d90e
Перевірка початкового повідомлення: PASS
Змінене повідомлення: Approve document 43
Повторна перевірка: InvalidSignature – відхилено, PASS

Підсумок: 4/4 контрольні перевірки пройдено.
(venv) ~/PycharmProjects/proj_1/1-2/Лабораторна_1_A_Комплект %

```

## Цифровий підпис

```

Python 3.12.0 | cryptography 50.0.1
Алгоритм: Ed25519
Повідомлення: Approve document 42
Закритий ключ: створено, не експортується
Відкритий ключ (32 байт):
9b793e8533139a7ea88dd5f44b159e9a0e9898d9ea742e1da1d01773974777f4
Підпис (64 байт), hex у двох рядках:
e84bbf23b843c6b9adbbadb0110082cd6f9c4106aea7390cf6fd3714a69f56bb
fc6ab8c9025a5eefab28bacfebb744eb15399d0effdcfc9d4a36af658682d90e
Перевірка початкового повідомлення: PASS
Змінене повідомлення: Approve document 43
Повторна перевірка: InvalidSignature — відхилено, PASS

```

### Рисунок 2 – Результат підписання та перевірки Ed25519

Для початкового повідомлення verify завершився без винятку. Після заміни номера документа 42 на 43 той самий підпис не пройшов перевірку: отримано InvalidSignature. Повідомлення змінено, але підпис залишено початковим, що демонструє виявлення підміни. У повному коді неочікуване прийняття змінених даних завершує програму помилкою.

Таблиця 2 – Підсумок контрольних перевірок

| Перевірка                              | Фактичний результат               |
|----------------------------------------|-----------------------------------|
| Розшифрування початкового шифротексту  | Початкове повідомлення відновлено |
| Розшифрування після зміни одного байта | InvalidTag; підміну відхилено     |

|                                                |                                     |
|------------------------------------------------|-------------------------------------|
| Перевірка початкового підписаного повідомлення | Підпис підтверджено                 |
| Перевірка зміненого повідомлення               | InvalidSignature; підміну відхилено |

Усі 4 контрольні перевірки пройдено. Це підтверджує коректність продемонстрованих викликів API та обробки двох видів підміни в цьому досліді; навантаження, фізичні атаки й повний захист IT-системи цими прикладами не перевірялися.

## 5 Вибір засобів для чотирьох IT-сценаріїв

### 5.1 Сценарій A Backend-сервіс

1. AEAD і зручна обробка помилок потрібні для конфіденційності даних та відхилення підмін. 2. Продуктивний нативний backend і доступне апаратне прискорення зменшують витрати CPU. 3. Унікальність nonce між процесами та після перезапуску запобігає небезпечному повторному використанню. 4. Інтеграція з KMS/HSM забезпечує централізований контроль ключів. 5. Підтримка та оперативні оновлення зменшують час експлуатації відомих вразливостей. Обраний засіб: cryptography для локальних операцій із окремим API системи керування ключами .

### 5.2 Сценарій Б Мобільний застосунок

Для Android: 1. AES-GCM захищає локальні дані та їх цілісність. 2. Android Keystore дає змогу використовувати ключ без експорту його матеріалу в процес застосунку. 3. Апаратне зберігання у TEE або StrongBox бажане за наявності підтримки конкретного алгоритму. 4. Вимога автентифікації користувача обмежує використання ключа після втрати пристрою. 5. Обробка втрати або інвалідації ключа потрібна для передбачуваної поведінки після зміни налаштувань безпеки. Обраний засіб: нативний JCA/JCE з провайдером AndroidKeyStore .

### 5.3 Сценарій В Сервіс цифрового підпису

1. Закритий ключ має бути неекспортованим у HSM або керованому сервісі підпису, оскільки його копіювання дозволяє створювати підроблені підписи. 2. Потрібне розмежування доступу до операції підписання. 3. Журналювання метаданих операцій дає змогу розслідувати зловживання. 4.

Алгоритм і формат підпису повинні підтримуватися сторонами обміну. 5. Відмовостійкість і процедура заміни ключа забезпечують роботу та реагування на компрометацію. Обраний засіб: API HSM/KMS, а бібліотека Python – для підготовки даних і перевірки; можливості залежать від провайдера.

#### 5.4 Сценарій Г Embedded та IoT

Для ESP32: 1. Мала витрата RAM і flash потрібна через обмеження пристрою. 2. Апаратне прискорення підтримуваних операцій скорочує час і енерговитрати. 3. Коректне джерело ентропії необхідне для ключів; слід виконати умови активації фізичної ентропії ESP32. 4. Підтримка AEAD і TLS захищає збережені дані та обмін. 5. Сумісність зі штатним SDK спрощує оновлення й конфігурацію. Обраний засіб: Mbed TLS в ESP-IDF.

Wokwi може використовуватися для перевірки логіки навчального прикладу, але емуляція не є доказом фізичної ентропії або стійкості апаратного захисту реального ESP32. Для цього сценарію в межах першої роботи виконано аналіз вимог; запуск емулятора не входить до двох контрольних прикладів.

### 6 Індивідуальне завдання Варіант А

#### 6.1 Вимоги до backend-сервісу

Сервіс обробляє конфіденційні дані багатьох користувачів і виконує значну кількість криптографічних операцій. Основні вимоги: захист даних під час зберігання та передавання; автентифіковане розшифрування; розділення доступу між користувачами; контроль життєвого циклу ключів; робота кількох процесів без повторів nonce; передбачувана затримка; підтримувані залежності. Конкретний норматив продуктивності встановлюють після визначення навантаження та перевіряють бенчмарком на цільовому сервері.

#### 6.2 Бібліотека та криптографічні функції

Основна бібліотека – cryptography. Для даних застосовано AES-256-GCM; для похідних ключів за потреби – HKDF із розділенням

контекстів; для автентичності службових повідомлень – HMAC-SHA-256. Ed25519 придатний для підписування документів, якщо таку функцію передбачає сервіс. Випадкові байти отримують через системний CSPRNG. Паролі, якщо вони є в системі, обробляють спеціальною password-KDF, а не зворотним шифруванням. Передавання даних захищає налаштований TLS-стек; сама cryptography не замінює TLS-сервер.

### 6.3 Організація ключів та зберігання даних

Пропонується конвертне шифрування: ключ даних DEK виконує локальне шифрування, а ключ KEK у KMS захищає DEK. До БД записують версію формату, ідентифікатор KEK, зашифрований DEK, nonce, шифротекст і tag. Відкритий DEK доступний лише авторизованому процесу під час операції; KEK залишається у сховищі. KMS викликають через його окремий SDK/API: cryptography сама по собі не є системою керування ключами.

Для уникнення координації nonce між усіма процесами базова схема створює свіжий DEK для кожного запису або його нової зашифрованої версії. Під цим DEK виконується одне шифрування з 12-байтовим nonce. Ідентифікатор користувача, запису й версії включають до однозначно серіалізованих AAD. Під час читання очікуваний контекст формують із авторизованого запиту, а не лише з недовірених полів БД, щоб виявляти перенесення шифротексту між записами.

Якщо вимірювання покажуть потребу повторно використовувати DEK, перед цим вводять окрему стійку до перезапусків схему nonce, наприклад виділені процесам неперетинні діапазони лічильника. Оновлення лічильника повинно бути атомарним; відновлення резервної копії не може повертати вже використані значення. Політику ротації узгоджують з обсягом шифрування та обраним режимом .

### 6.4 Можливості програмно-апаратної платформи

Доцільні KMS/HSM, ідентичності сервісів із короткочасними обліковими даними, мінімальні дозволи на розкриття DEK, аудит операцій, розмежування адміністраторів БД і ключів. Апаратне прискорення

використовують там, де воно підтримується backend і CPU. Секрети не записують у журнали, репозиторій або відкриті конфігурації. Обмежують доступ до дамів пам'яті та зменшують час життя відкритого DEK.

Python не гарантує надійного стирання всіх копій секрету в пам'яті: незмінні bytes можуть залишатися після завершення логічного використання. Тому del не вважається гарантованим знищенням ключа. Якщо модель загроз вимагає, щоб приватний ключ узагалі не потрапляв до процесу, операцію виконують у HSM/KMS із неекспортованим ключем .

### 6.5 Обґрунтування вибору

cryptography покриває потрібні локальні операції, підтримує AEAD із перевіркою tag і надає окремі винятки для відмови автентифікації. Нативний backend є придатною основою для продуктивного сервісу, а політика security-оновлень дозволяє організувати супровід. KMS доповнює бібліотеку керування ключами. За вимоги FIPS обирають перевірену конфігурацію валідованого модуля та дозволені операції; навчальну інсталяцію не оголошують сертифікованою .

### 6.6 Ризики неправильного використання API

Повтор попсо може розкрити зв'язок між повідомленнями та дозволити підроблення. Ігнорування InvalidTag перетворює автентифікацію на формальність. Неправильний AAD допускає перестановку записів між контекстами. Витік DEK через журнали або дампи розкриває відповідні дані. Надмірні права на KMS дозволяють зловмиснику розшифровувати дані навіть без експорту KEK. Прийняття недовіреного відкритого ключа нівелює перевірку підпису. Усунення ризиків потребує перевірки контексту, мінімальних прав, ротації та оновлень.

AEAD не захищає від повторного подання цілого раніше справжнього запису. Для протидії replay/rollback сервіс перевіряє актуальну версію або стан операції в довіреному сховищі. Шифрування БД також не захищає від повністю скомпрометованого авторизованого backend-процесу, який уже має право одержувати відкриті дані. Запропонована архітектура є результатом

проєктного аналізу; реалізація повної ІТ-системи для індивідуального завдання не вимагалася.

## 7 Висновки

Порівняно cryptography та PyCryptodome за всіма критеріями завдання. Для Python backend-сервісу обрано cryptography завдяки відповідності потрібним операціям, зручному API та процесу підтримки. У практичних прикладах AES-GCM відновив початкове повідомлення й відхилив змінений шифротекст, а Ed25519 підтвердив початковий підпис та виявив зміну повідомлення. Усі чотири контрольні перевірки пройдено.

Аналіз чотирьох сценаріїв показав різні пріоритети: для backend важливі керування ключами та паралельна робота, для мобільного застосунку – нативне захищене сховище, для сервісу підпису – неекспортований ключ, для IoT – ресурси та ентропія. Для варіанта А запропоновано конвертне шифрування з KMS, перевіркою AAD і контрольованим використанням nonce. Коректний вибір алгоритму має доповнюватися контролем доступу, обробкою помилок та супроводом залежностей. –